

Project Executable Files

Date	2 July 2026
Team ID	
Project Name	Voice-Based Concept Understanding Analyser (VBCUA)
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S. No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA (no database used — reference data stored in JSON)
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA (web application, not mobile/desktop)
8	Dockerfile / Containerization config (if applicable)	Yes

9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```

/project-root
/src
  streamlit_app.py      - Main Streamlit application
  /modules              - Core logic modules
    transcription.py    - Whisper speech-to-text
    semantic_analysis.py - Sentence-BERT similarity scoring
    audio_features.py   - Librosa fluency analysis
    scoring.py          - Combined scoring engine
    feedback_generator.py - Gemini AI feedback generation
    report_generator.py - PDF report generation
  /data
    reference_concepts.json - Reference concept explanations
  /sample_audio
    test.m4a            - Sample test recording
Dockerfile              - Container build configuration
requirements.txt        - Python dependencies
README.md               - Setup and run instructions
.env.example            - Environment variable template (Gemini API key)
  
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://malfoydraco-vbcua.hf.space/
Login Credentials (Demo)	NA — no login required, publicly accessible
Platform / Hosting Provider	Hugging Face Spaces (Docker SDK, CPU Basic — free tier)
Repository Link	https://github.com/PudiBhargavi/VBCUA.git
Demo Video Link	https://youtu.be/ArJnygVXp0

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

1. Clone the repository: `git clone <your-github-repo-url>`
2. Navigate to the src folder: `cd project-root/src`
3. Create a virtual environment: `python -m venv venv`
4. Activate it: `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Mac/Linux)
5. Install dependencies: `pip install -r requirements.txt`
6. Install ffmpeg (required by Whisper for audio processing) and ensure it's on your system PATH
7. Create a .env file in the src folder with your Gemini API key:
`GEMINI_API_KEY=your_key_here`
8. Run the application: `streamlit run streamlit_app.py`
9. Open browser at <http://localhost:8501>

pre-requisites:

- Python 3.10 or higher
- ffmpeg installed and added to system PATH
- A Google Gemini API key (free tier available via Google AI Studio)
- Minimum 4GB RAM recommended (for running Whisper and Sentence-BERT models locally)

Step 5: Known Issues / Limitations

S. No	Known Issue / Limitation	Workaround / Status
1	Hugging Face free-tier Space sleeps after 48 hours of inactivity, causing a slower first response (cold start)	Visit the deployed link periodically to keep it active; resolved by simply waiting ~30-60 sec on first load
2	Gemini API free tier has a rate limit of 5 requests/minute	Implemented automatic retry logic with fallback message to handle this gracefully
3	Whisper "base" model may slightly mis transcribe uncommon technical terms	Acceptable trade-off for free, CPU-friendly transcription; semantic scoring is resilient to minor transcription noise